

- 1) ¿Cuál es el tamaño de la componente conexa más grande? ¿Cuántas componentes conexas hay?

Hay 2746 componentes conexas.

El tamaño de la componente conexa más grande es de 855802 vértices.

- 2) Calcular el camino mínimo de todos con todos. ¿En cuánto tiempo lo puede hacer? ¿Qué orden tiene el algoritmo? En caso de no alcanzarle el tiempo, estime cuanto tiempo le llevaría.

Como mi grafo tiene tantos vértices, no voy a poder calcular el tiempo que me llevaría recorrer todos los caminos mínimos; pero podría tomar una muestra de tamaño N y calcular el promedio de los caminos mínimos de esa muestra multiplicado por la cantidad total de nodos que tenemos.

Considerando esto, con una muestra de 100 nodos el algoritmo tomaría aproximadamente: 328hs, 43 minutos con 20.9 segundos.

La complejidad del algoritmo de cálculo de caminos mínimos (BFS) es de $O(|V| + |E|)$ para un solo nodo. Eso, si lo multiplicamos por todos los nodos del grafo nos quedará una complejidad de $O[|V| * (|V| + |E|)]$, por lo que en nuestro grafo no terminaría nunca.

Por este motivo, al realizar una aproximación, nuestra complejidad pasa a ser simplemente de $O[|N| * (|V| + |E|)]$, donde N es la cantidad de nodos que tomamos como muestra, los cuales son elegidos de forma aleatoria.

- 3) ¿Cuántos triángulos tiene el grafo?

Podemos dividirlo en triángulos contados con el grafo dirigido y no dirigido.

- Cantidad de triángulos con el grafo dirigido: 3,889,771
- Cantidad de triángulos con el grafo no dirigido: 13,391,903

- 4) Utilizando el punto 2, calcule el diámetro del grafo

Nuevamente, no puedo calcular el diámetro de todo el grafo por los mismos motivos que expliqué en el punto 2.

Tomando 10 nodos de referencia e iterando por todos y “rebotando” hasta llegar al camino máximo, logro obtener que el diámetro estimado del grafo es de 24 aristas.

5) Calcule el pageRank del grafo

Los 10 nodos con mayor PageRank son:

```
# Results
# Converged after 61 iterations, aborting...
# Top vertices with the highest PageRank:
# Vertex: 597621, PageRank: 0.000644356692909492
# Vertex: 41909, PageRank: 0.0006425477256693725
# Vertex: 163075, PageRank: 0.0006305999262383353
# Vertex: 537039, PageRank: 0.0006269920571772538
# Vertex: 384666, PageRank: 0.0005489073817720516
# Vertex: 504140, PageRank: 0.0005337169626427533
# Vertex: 486980, PageRank: 0.000505691228595806
# Vertex: 605856, PageRank: 0.0005008187129537594
# Vertex: 32163, PageRank: 0.0004970646588593843
# Vertex: 558791, PageRank: 0.0004947016865199656
```

6) Calcule la circunferencia del grafo

Nuevamente esto es imposible considerando lo grande que es el grafo.

Por ese mismo motivo, el algoritmo que utilicé para aproximar la circunferencia del grafo fue hacer una especie de búsqueda binaria de ciclos de largo N recortados a un determinado valor de timeout, donde, si en un tiempo determinado no logra encontrar el ciclo, considera que no existe.

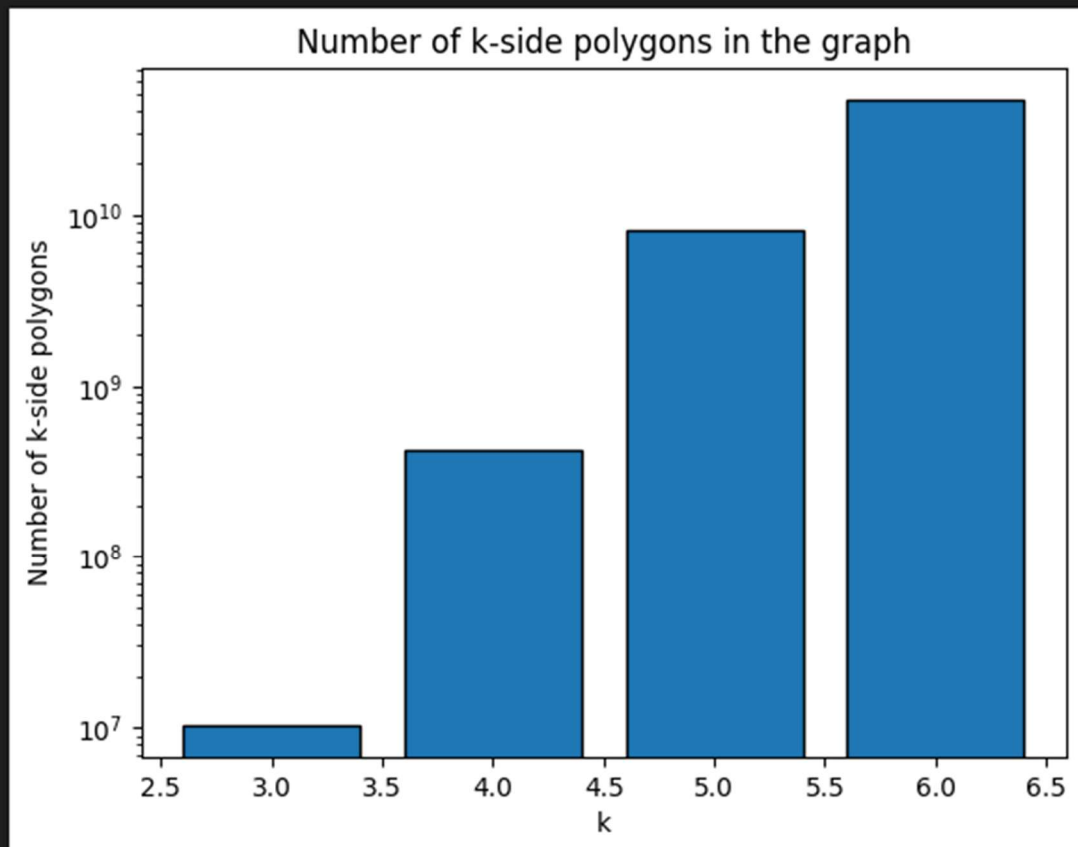
```
# Estimating the graph's circumference...
#       Cycle of length 214 found!
#       Cycle of length 321 found!
#       Cycle of length 327 found!
#       Cycle of length 330 found!
#       Cycle of length 331 found!
# Estimated circumference of the graph: 331
# Time elapsed: 1m 37.0s
```

Ejercicios adicionales

Adicional 1: Esta función estima los polígonos de tamaño K. Tarda una eternidad y encima estoy aproximando. Realizo 100 iteraciones para ciclos de 3, 50 para ciclos de 4, 25 para ciclos de 5 y 25 para ciclos de 6, por lo que todos los valores son estimadores.

NOTA: En el gráfico se ve que $K = 3$ está sobreestimado casi 10 veces de su valor real. Esto está solucionado en el código haciendo una pequeña verificación que dice que si K es = 3 simplemente busque los triángulos que existen en el grafo no dirigido, pero debido al tiempo que toma hacer este gráfico no lo actualicé.

```
Reading web-Google.txt...
100%|██████████| 5105039/5105040 [00:07<00:00, 696877.69it/s]
Finished reading web-Google.txt
-----
Extra1
-----
Estimating 3-cycles: 100%|██████████| 100/100 [07:59<00:00, 4.79s/it]
Estimating 4-cycles: 100%|██████████| 50/50 [04:31<00:00, 5.43s/it]
Estimating 5-cycles: 100%|██████████| 25/25 [08:43<00:00, 20.94s/it]
Estimating 6-cycles: 100%|██████████| 10/10 [2:04:45<00:00, 748.57s/it]
```



Adicional 2: Cálculo del coeficiente de clustering del grafo:

Esto se podría realizar de dos formas, de forma dirigida y no dirigida. Como no sabía cuál calcular hice ambas.

- Average Clustering Coefficient no dirigido: 0.5142961475354295
- Average Clustering Coefficient dirigido: 0.3651263215748557

Adicional 3: Betweenness centrality del grafo.

Nuevamente, esto implicaría recorrer todo el grafo con el punto 2, y tardaría más de 300 horas. Por ese motivo tomamos N simples (En este caso 100) y el resultado es:

```
# Results
# Node with the highest betweenness centrality: 560622, Value: 48932.75
# Time elapsed: 3m 42.17s
```